

Chapter 14: Movement and Resizing

Moving and resizing objects on a form should be second nature to any seasoned C# programmer. Now I won't suggest that C# is the preferred language for gamers, but it is certainly the preferred language for everything that happens around the actual animations.

Given the importance of movement and resizing in everyday programming let's jump to the inside and look at some code.

We begin of course by creating a new Windows Forms project entitled Chapter 14 Movement and Resizing.

I began by adding a Timer, Button and PictureBox. With those objects in place, I set the following properties:

Form1		
	Size	1200,600
	Text	Movement and Resizing

timer1		
	Interval	100

button1		
	Name	StartStopButton
	Text	Start
	Size	75,25
	Location	1100,10

pictureBox1		
	Image	Any you like
	SizeMode	StretchImage

It is now time to write some code and make this stuff move and resize. We'll start with resizing and the Form1_Load routine shown here. (Figure 14.1) As you can see, I began by defining some enums named Directions, Controller and Changing. I then instantiated an instance of each. The FormHeight and FormWidth variables are going to be used to limit the movement of our onscreen object and the x_location and y_location variables are going to be the upper left-hand pixel location of our object. We'll use the ran variable a little later in this chapter so stay tuned on that guy.

Figure 14.1: Form1_Load

```
namespace chapter_14_v2
{
    3 references
    public partial class Form1 : Form
    {
        2 references
        enum Directions { Up, Down, Left, Right, Clockwise, Counterclockwise, Bigger, Smaller };
        2 references
        enum Controller { Button, Keyboard, MouseLeft };
        2 references
        enum Changing { Size, Location, Rotation };
        Directions direction;
        Controller controller;
        Changing changing;
        int FormHeight = 600, FormWidth = 1200, x_location = 525, y_location = 225;
        Random ran = new Random();
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void Form1_Load(object sender, EventArgs e)
        {
            // in our initial example we will adjust the size of our image
            direction = Directions.Bigger;
            controller = Controller.Button;
            changing = Changing.Size;

            StartStopButton.Text = "Start";
            timer1.Enabled = false;

            pictureBox1.Location = new Point(x_location, y_location);
            pictureBox1.Size = new Size(50, 50);
        }
    }
}
```

Next came the four routines shown here. (Figure 14.2) In our first example we are going to resize our PictureBox with every Tick of the timer. On each Tick we will call one of the ChangeSize routines based on the value of our direction variable. Not surprisingly, our ChangeSize routines continue to resize our object until it reaches the size of the form or until it is minimally small. Once those boundary values are met the value of our direction variable is changed to its opposite value. The StartStopButton_Click routine is pretty self-explanatory, manage the timer and the Text property of the Button.

Figure 14.2: ChangeSize Routines

```
1 reference
private void timer1_Tick(object sender, EventArgs e)
{
    if (changing == Changing.Size) // we are going to resize our pictureBox
    {
        pictureBox1.Location = new Point(10, 10);
        if (direction == Directions.Bigger) ChangeSizeBigger();
        else ChangeSizeSmaller();
    }
}

1 reference
private void ChangeSizeBigger()
{
    if (pictureBox1.Height < FormHeight && pictureBox1.Width < FormWidth)
        pictureBox1.Size = new Size(pictureBox1.Width + 20, pictureBox1.Height + 8);
    else
        direction = Directions.Smaller;
}

1 reference
private void ChangeSizeSmaller()
{
    if (pictureBox1.Height > 10 && pictureBox1.Width > 10)
        pictureBox1.Size = new Size(pictureBox1.Width - 20, pictureBox1.Height - 8);
    else
        direction = Directions.Bigger;
}

1 reference
private void StartStopButton_Click(object sender, EventArgs e)
{
    if (StartStopButton.Text == "Start")
    {
        timer1.Enabled = true;
        StartStopButton.Text = "Stop";
    }
    else
    {
        timer1.Enabled = false;
        StartStopButton.Text = "Start";
    }
}
```

We are next going to move our image randomly about the Form. Once the image starts moving in a particular direction it will continue to do so until it meets the edge of the opposite side of the Form. At that point we will randomly pick another direction and start the movement again. We will again start and stop the movements via the Button.

The updated Form_Load routine and our new PickDirection routines are shown here. (Figure 14.3) Come on, that PickDirection is pretty darn elegant. Not a lot of wasted code in that guy.

Figure 14.3: PickDirection

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    // in our initial example we will adjust the size of our image
    direction = Directions.Left;
    controller = Controller.Button;
    changing = Changing.Location;

    if (changing == Changing.Location) PickDirection();
    StartStopButton.Text = "Start";
    timer1.Enabled = false;

    pictureBox1.Location = new Point(x_location, y_location);
    pictureBox1.Size = new Size(50, 50);
}

1 reference
private void PickDirection()
{
    int r = ran.Next(4); // 0=left, 1=right, 2=up, 3=down
    if (r == 0) direction = Directions.Left;
    else if (r == 1) direction = Directions.Right;
    else if (r == 2) direction = Directions.Up;
    else if (r == 3) direction = Directions.Down;
}
```

The ChangeLocation routine shown here is also super straightforward. (Figure 14.4) Check which direction the image is currently moving and if there is space enough to make another step in that direction then make the move, else go ahead and pick a new direction with the PickDirection routine. Question, what happens if the next direction picked is the same as the currently maxed out one? No problem right, we just go back to the PickDirection routine until you get a new direction.

Figure 14.4: ChangeLocation

```
0 references
private void ChangeLocation()
{
    if (direction == Directions.Left)
    {
        if (x_location > 10)
        {
            x_location -= 10;
            pictureBox1.Left = x_location;
        }
        else PickDirection();
    }
    else if (direction == Directions.Right)
    {
        if (x_location + 10 < FormWidth - 100)
        {
            x_location += 10;
            pictureBox1.Left = x_location;
        }
        else PickDirection();
    }
    else if (direction == Directions.Up)
    {
        if (y_location > 10)
        {
            y_location -= 10;
            pictureBox1.Top = y_location;
        }
        else PickDirection();
    }
    else if (direction == Directions.Down)
    {
        if (y_location < FormHeight - 100)
        {
            y_location += 10;
            pictureBox1.Top = y_location;
        }
        else PickDirection();
    }
}
}
```

We of course also needed to slightly modify our Form1_Load and timer1_Tick routines. The updated routines are shown here. (Figure 14.5)

Figure 14.5: Modified Load and Tick Routines

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    // we will start by resizing our image and starting/stopping via button
    direction = Directions.Bigger;
    controller = Controller.Button;
    //changing = Changing.Size;
    changing = Changing.Location;

    if (changing == Changing.Location) PickDirection();

    StartStopButton.Text = "Start";
    timer1.Enabled = false;

    pictureBox1.Location = new Point(x_location, y_location);
    pictureBox1.Size = new Size(50, 50);
}

5 references
private void PickDirection()
{
    int r = ran.Next(4); // 0 = Left, 1 = Right, 2 = Up, 3 = Down
    if (r == 0) direction = Directions.Left;
    else if (r == 1) direction = Directions.Right;
    else if (r == 2) direction = Directions.Up;
    else direction = Directions.Down;
}

1 reference
private void timer1_Tick(object sender, EventArgs e)
{
    if (changing == Changing.Size) // we are going to resize our pictureBox
    {
        pictureBox1.Location = new Point(10, 10); // move pictureBox to upper left-hand corner
        if (direction == Directions.Bigger) ChangeSizeBigger();
        else ChangeSizeSmaller();
    }
    else if (changing == Changing.Location)
    {
        pictureBox1.Location = new Point(10, 10); // move pictureBox to upper left-hand corner
        ChangeLocation();
    }
}
```

Time to leave our Start StopButton and start moving our image around the Form using the arrow keys. We begin that journey with the small modifications shown here to our Load routine. (Figure 14.6)

Figure 14.6: Updated Load Routine

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    direction = Directions.Bigger;
    controller = Controller.Keyboard;
    changing = Changing.Location;

    if (changing == Changing.Location) PickDirection();

    StartStopButton.Text = "Start";

    if (controller == Controller.Button)
    {
        timer1.Enabled = false;
        StartStopButton.Visible = true;
    }
    else if (controller == Controller.Keyboard)
    {
        timer1.Enabled = true;
        StartStopButton.Visible = false;
    }

    pictureBox1.Location = new Point(x_location, y_location);
    pictureBox1.Size = new Size(50, 50);
}
```

The truly magical code for managing the arrow keys is shown next. (Figure 14.7) The top portion of the image shows a modified ChangeLocation procedure. I added a conditional at the top which checked whether our controller variable was set to Button and if it was we executed the old code. If it was set to Keyboard then we change our x and y_location variables to the value of our direction variable. We also of course check that the user does not run the image off the form.

The cool procedure is the one named Form1_KeyDown. The first thing to note is that the e parameter is actually of class type KeyEventArgs rather than just the regular EventArgs class. The KeyEventArgs class carries the various keyboard properties and methods. So how do we use it? As shown in the example, the most common property is KeyCode which identifies which key on the keyboard was pressed. Compare that value to Visual Studio's enum of Keys and we are in business. So cool and super easy once you see it in an example.

By the way, don't forget to attach our Form1_KeyDown routine to the Form's KeyDown event in the Properties pane.

Figure 14.7: Form1_KeyDown

```
    {
        if (y_location < FormHeight - 100)
        {
            y_location += 10;
            pictureBox1.Top = y_location;
        }
        else PickDirection();
    }

    else if (controller == Controller.Keyboard)
    {
        if (direction == Directions.Left && x_location >= 20) x_location -= 10;
        else if (direction == Directions.Right && x_location < FormWidth - 50 - 20) x_location += 10;
        else if (direction == Directions.Up && y_location >= 20) y_location -= 10;
        else if (direction == Directions.Down && y_location <= FormHeight - 50 - 20) y_location += 10;

        pictureBox1.Location = new Point(x_location, y_location);
    }
}

0 references
private void Form1_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Left) direction = Directions.Left;
    else if (e.KeyCode == Keys.Right) direction = Directions.Right;
    else if (e.KeyCode == Keys.Up) direction = Directions.Up;
    else if (e.KeyCode == Keys.Down) direction = Directions.Down;
}
}
```

What if we had wanted to move just a single step with each keyboard press? Easy, we would have simply moved the else if (controller == Controller.Keyboard) logic in the above example down into the Form1_KeyDown routine.

Moving our image to the location of a mouse click is equally straightforward. We begin by updating our Load code to set our controller variable equal to a left mouse click event. Also please note that we don't need our Timer for this condition as all events are triggered off of a mouse click. (Figure 14.8)

Figure 14.8: Update Form1_Load

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    direction = Directions.Bigger;
    controller = Controller.MouseLeft;
    changing = Changing.Location;

    if (changing == Changing.Location && controller == Controller.Button) PickDirection();

    StartStopButton.Text = "Start";

    if (controller == Controller.Button)
    {
        timer1.Enabled = false;
        StartStopButton.Visible = true;
    }
    else if (controller == Controller.Keyboard)
    {
        timer1.Enabled = true;
        StartStopButton.Visible = false;
    }
    else if (controller == Controller.MouseLeft)
    {
        StartStopButton.Visible = false;
        timer1.Enabled = false;
    }

    pictureBox1.Location = new Point(x_location, y_location);
    pictureBox1.Size = new Size(50, 50);
}
```

The Form1_MouseDown routine shown here is also very straightforward. (Figure 14.9) All we do is check that we are working off of the correct mouse button and then set the Top and Left properties of our image to those of the event e parameter which is of class type MouseEventArgs. Again, very easy yet dead on effective. Anybody want to guess how we would change this code to handle a right mouse button click? Of course, just check for a different value of the MouseButtons enum.

Again, please don't forget to assign this procedure to your form's MouseDown event.

Figure 14.9: Form1_MouseDown

```
1 reference
private void Form1_MouseDown(object sender, MouseEventArgs e)
{
    // jump to the location of mouse at time of left button click
    if (e.Button == MouseButtons.Left && controller == Controller.MouseLeft)
    {
        pictureBox1.Top = e.Y;
        pictureBox1.Left = e.X;
    }
}
5 references
```

Our next movement, rotation, is super easy because Visual Studio provides us with a RotateFlip method for the Image class. This figure shows us the minor updates I made to the Form1_Load routine. (Figure 14.10) Not much going on here. I loaded a wheel image from a hard drive and swapped it into the pictureBox1 object and then I increased the size of that same PictureBox. Yes, I chose the wheel image to increase the rotation's visual impact.

Figure 14.10: Updated Form1_Load

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    direction = Directions.Bigger;
    controller = Controller.MouseLeft;
    changing = Changing.Rotation;

    if (changing == Changing.Location && controller == Controller.Button) PickDirection();

    StartStopButton.Text = "Start";

    if (changing == Changing.Rotation)
    {
        pictureBox1.Size = new Size(500, 500); // make it a little bigger so it's easier to see the rotation
        Image img1 = Image.FromFile(@"F:\documents\C# The Whole Story\Section 3 GUI Stuff\Chapter 14 Movement\images\wheel image.jpg");
        pictureBox1.Image = img1;
        StartStopButton.Visible = false;
        timer1.Enabled = true;
    }
    else if (controller == Controller.Button)
    {
        timer1.Enabled = false;
        StartStopButton.Visible = true;
    }
    else if (controller == Controller.Keyboard)
    {
        timer1.Enabled = true;
        StartStopButton.Visible = false;
    }
    else if (controller == Controller.MouseLeft)
    {
        StartStopButton.Visible = false;
        timer1.Enabled = false;
    }

    pictureBox1.Location = new Point(x_location, y_location);
    pictureBox1.Size = new Size(50, 50);
}
```

This image shows the modifications I made to the timer1_Tick event handler and a new routine I created aptly named SpinSomething. (Figure 14.11) Check out all the various values available via the RotateFlipTypes enum. You will basically find all combinations of angle of rotation and axis of flip.

Figure 14.11: SpinSomething

```
1 reference
private void timer1_Tick(object sender, EventArgs e)
{
    if (changing == Changing.Size) // we are going to resize our pictureBox
    {
        pictureBox1.Location = new Point(10, 10); // move pictureBox to upper left-hand corner
        if (direction == Directions.Bigger) ChangeSizeBigger();
        else ChangeSizeSmaller();
    }
    else if (changing == Changing.Location)
    {
        pictureBox1.Location = new Point(10, 10); // move pictureBox to upper left-hand corner
        ChangeLocation();
    }
    else if (changing == Changing.Rotation) SpinSomething();
}

1 reference
private void SpinSomething()
{
    Image img = new Bitmap(pictureBox1.Image);
    img.RotateFlip(RotateFlipType.Rotate90FlipNone);
    pictureBox1.Image = img;
}

1 reference
private void ChangeSizeBigger()
{
    if (pictureBox1.Height < FormHeight && pictureBox1.Width < FormWidth)
        pictureBox1.Size = new Size(pictureBox1.Width + 20, pictureBox1.Height + 8);
    else
        direction = Directions.Smaller;
}
```

This concludes our discussion of movement and resizing. In our next chapter we will look at a very simple online learning and testing application. Something obviously very close to my own heart.